CLOUD-BASED NETWORK INTRUSION DETECTION SYSTEM

GitHub & Render Deployment Guide

Project	Design and Implementation of Network Intrusion Detection Using Cloud Computing
Frontend	React + TypeScript + Vite
Backend	FastAPI + SQLAlchemy
Database	PostgreSQL on Render
Regional time	Africa/Lagos · West Africa Time (UTC+1)
Deployment	GitHub repository + Render Blueprint (render.yaml)

This guide accompanies the source-code ZIP. It explains how to publish the project to GitHub and deploy its frontend, API and database through Render.

1. What is included

The repository is configured as a three-part cloud deployment. The React dashboard is built into static files and served by Render. The FastAPI service exposes protected APIs and receives events. PostgreSQL stores users, traffic events, alerts and audit records.

- Frontend service: **nids-frontend** — Render Static Site.
- Backend service: **nids-api** — Render Python Web Service.
- Database: **nids-postgres** — Render PostgreSQL.
- Render Blueprint: **render.yaml** at the repository root.
- Regional configuration: timestamps are stored in UTC and rendered in **Africa/Lagos**.

2. Publish the source code to GitHub

- Extract **network-intrusion-detection-cloud.zip** on your computer.
- Create a new GitHub repository, for example **cloud-network-intrusion-detection**.
- Upload the contents of the extracted project folder to the repository root. Do not upload the enclosing folder as an extra nested directory.
- Confirm that the root of the repository contains **render.yaml**, **backend/**, **frontend/**, **sensor/** and **docs/**.
- Do not commit .env files, passwords, database files, node_modules or virtual environments.

Alternative: Git commands

```
git init
git add .
git commit -m "Initial cloud NIDS implementation"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/YOUR-REPOSITORY.git
git push -u origin main
```

3. Deploy with Render Blueprint

- Sign in to Render and connect your GitHub account.
- Choose **New +** → **Blueprint**.
- Select the GitHub repository and the branch containing render.yaml.
- Review the proposed resources: nids-api, nids-frontend and nids-postgres.
- When prompted, enter your administrator email and a unique password of at least 12 characters. Do not use the local demo password.
- Confirm the resource creation and wait for the database, API and static frontend deployments to complete.

Cost warning: the included Blueprint selects Render Postgres plan **basic-256mb**, which is a paid database plan. The API and static site are set to free plans, subject to Render's free-service restrictions. Check Render's current pricing before creating resources.

4. Default service addresses

Purpose	Default address
Frontend	https://nids-frontend.onrender.com
Backend API	https://nids-api.onrender.com

Health check	https://nids-api.onrender.com/api/health
Interactive API docs	https://nids-api.onrender.com/docs

These addresses assume the default service names in render.yaml. If you rename a service or Render assigns a different hostname, update the environment variables below.

Environment variables if URLs change

- On the frontend service, set **VITE_API_URL** to the exact backend URL, without a trailing slash, then redeploy the static site.
- On the API service, set **CORS_ORIGINS** to the exact frontend origin (scheme and hostname only), then redeploy the API.

5. Sign in and verify the system

- Open the frontend URL.
- Sign in using the ADMIN_EMAIL and ADMIN_PASSWORD entered during Blueprint creation.
- Open Simulation Lab and run Port Scan, Brute Force, SYN Flood Indicator, Suspicious DNS and Normal Traffic.
- Open Security Alerts and confirm that generated alerts are listed.
- Open Live Monitoring and inspect the stored traffic metadata.
- Export a CSV report from Security Alerts.
- Open the API health URL and confirm that the returned timezone is Africa/Lagos.

6. Local development

For a local Docker run, install Docker Desktop, open a terminal in the project root and execute:

```
docker compose up --build
```

Open http://localhost:5173 for the dashboard and http://localhost:8000/docs for API documentation. Stop with Ctrl+C. To remove local containers and the local database volume, run docker compose down -v; this deletes that local database data.

7. Troubleshooting

- **Frontend cannot reach API:** verify VITE_API_URL and CORS_ORIGINS. CORS_ORIGINS must match the frontend origin exactly.
- **Login fails:** check the administrator credentials entered during Blueprint creation. Changing ADMIN_PASSWORD later does not reset an already-created user's password.
- **Database connection error:** confirm the Render PostgreSQL service is healthy and DATABASE_URL references its internal connection string.
- **First API request is slow:** free web services may spin down while idle and take time to start again.
- **Records disappear:** use the managed PostgreSQL service for deployed data; do not depend on the container filesystem for persistence.
- **Build failure:** inspect the relevant Render deploy logs. Frontend build: npm install && npm run build. Backend build: pip install -r requirements.txt.

8. Security checklist

- Use a unique administrator password and keep JWT_SECRET and SENSOR_TOKEN in Render environment variables only.

- Keep CORS restricted to the deployed frontend origin.
- Use HTTPS for remote sensor ingestion; never place the sensor token in frontend code.
- Rotate any credential that may have been exposed.
- Monitor only networks for which you have explicit authorization.
- Before production use, add login rate limiting, role-based access control, database migrations, backups, monitoring and a security review.

9. Academic scope and limitations

The included detector uses explainable heuristic rules. The Simulation Lab creates synthetic records and does not transmit attack traffic. The optional Scapy sensor forwards packet metadata only. Repeated failed-login detection requires authentication failure events from service logs; ordinary packet capture cannot establish that a login failed. SYN-rate activity is an indicator, not proof of a denial-of-service attack.

This delivery does not include a trained machine-learning model or validated model accuracy. Do not claim ML performance in the report or defence unless a model has been trained and evaluated on a documented dataset. A defensible extension should document dataset provenance, feature processing, train/test separation, precision, recall, F1 score and confusion matrix.

10. Official Render references

Blueprint specification: https://render.com/docs/blueprint-spec

Static sites: https://render.com/docs/static-sites

Environment variables: https://render.com/docs/configure-environment-variables

Web services: https://render.com/docs/web-services

Prepared for the academic project: Design and Implementation of Network Intrusion Detection Using Cloud Computing.